

Design Patterns & Core Software Engineering Concepts

Interview Notes for Freshers

1. What are Design Patterns?

Definition

Design Patterns are reusable solutions to commonly occurring problems in software design.

They are not complete code, but a template or best practice that developers can use while building applications.

Think of them like:

- A common blueprint
- Proven coding techniques
- Standard ways to solve problems

Why Design Patterns are Important?

- Improve code readability
- Make code reusable
- Reduce code duplication
- Make applications scalable
- Help teams follow standard practices
- Easier maintenance

Real-Life Example

Imagine building houses. You do not redesign: doors, windows, staircases every time from scratch.

Similarly, in software: developers reuse common solutions called Design Patterns.

Types of Design Patterns

1. Creational Patterns

Used for object creation.

Examples: Singleton, Factory, Builder

2. Structural Patterns

Used for class/object structure.

Examples: Adapter, Decorator

3. Behavioral Patterns

Used for communication between objects.

Examples: Observer, Strategy

2. Difference Between Factory and Abstract Factory

Feature	Factory Pattern	Abstract Factory Pattern
Purpose	Creates one object	Creates families of related objects

Complexity	Simple	More complex
Focus	Single product	Multiple related products
Example	Create one car	Create car + engine + tyre
Level	Lower level	Higher level

Factory Pattern

Definition

Factory Pattern creates objects without exposing object creation logic to the client.

Example

Step 1: Interface

```
interface Animal {  
    void sound();  
}
```

Step 2: Implementations

```
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat implements Animal {  
    public void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

Step 3: Factory Class

```
class AnimalFactory {  
  
    public Animal getAnimal(String type) {  
  
        if(type.equals("DOG")) {  
            return new Dog();  
        }  
  
        else if(type.equals("CAT")) {  
            return new Cat();  
        }  
  
        return null;  
    }  
}
```

Step 4: Client Code

```
public class Main {
```

```

public static void main(String[] args) {

    AnimalFactory factory = new AnimalFactory();

    Animal a1 = factory.getAnimal("DOG");
    a1.sound();

    Animal a2 = factory.getAnimal("CAT");
    a2.sound();

}
}

```

Output

```

Dog barks
Cat meows

```

Abstract Factory Pattern

Definition

Abstract Factory creates groups of related objects.

Real-Life Example

A car company creates: Car, Engine, Tyres. All related products come from one factory.

Structure

Factory -> Creates multiple related objects

Example: GUI Factory (Windows Button/Checkbox, Mac Button/Checkbox)

3. Singleton Pattern

Definition

Singleton Pattern ensures: Only ONE object of a class is created and Global access to that object.

Why Use Singleton?

Used where only one instance is needed: Logger, Database Connection, Configuration Manager, Cache.

Singleton Implementation

```

class Singleton {

    private static Singleton obj;

    // Private constructor
    private Singleton() {
    }

    // Public method
    public static Singleton getInstance() {

        if(obj == null) {
            obj = new Singleton();
        }

        return obj;
    }
}

```

```
}  
}
```

Usage

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Singleton s1 = Singleton.getInstance();  
        Singleton s2 = Singleton.getInstance();  
  
        System.out.println(s1 == s2);  
    }  
}
```

Output

```
true
```

Both references point to same object.

4. Thread-Safe Singleton

Problem in Normal Singleton

In multithreading: Two threads may create two objects simultaneously.

Thread-Safe Singleton Solution

```
class Singleton {  
  
    private static Singleton obj;  
  
    private Singleton() {  
    }  
  
    public static synchronized Singleton getInstance() {  
  
        if(obj == null) {  
            obj = new Singleton();  
        }  
  
        return obj;  
    }  
}
```

Why synchronized?

Only one thread can enter method at a time. Prevents multiple object creation.

Best Optimized Singleton (Interview Favorite)

Double Checked Locking

```
class Singleton {  
  
    private static volatile Singleton obj;
```

```

private Singleton() {
}

public static Singleton getInstance() {

    if(obj == null) {

        synchronized(Singleton.class) {

            if(obj == null) {
                obj = new Singleton();
            }
        }
    }

    return obj;
}
}

```

5. What is Loose Coupling?

Definition

Loose coupling means: Classes depend less on each other. Changes in one class should not heavily affect another class.

Tight Coupling Example ❌

```

class Engine {

    void start() {
        System.out.println("Engine Started");
    }
}

class Car {

    Engine engine = new Engine();

    void drive() {
        engine.start();
    }
}

```

Problem: Car is directly dependent on Engine class

Loose Coupling Example ✅

```

interface Engine {

    void start();
}

class PetrolEngine implements Engine {

    public void start() {

```

```

        System.out.println("Petrol Engine Started");
    }
}

class DieselEngine implements Engine {

    public void start() {
        System.out.println("Diesel Engine Started");
    }
}

class Car {

    Engine engine;

    Car(Engine engine) {
        this.engine = engine;
    }

    void drive() {
        engine.start();
    }
}

```

Benefits of Loose Coupling: Easy testing, Flexible code, Easy maintenance, Easy replacement of modules.

6. Explain Dependency Injection (DI)

Definition

Dependency Injection means: Instead of creating objects inside a class, dependencies are provided from outside.

Simple Meaning: Class should not create dependency itself. Dependency should be injected externally.

Without Dependency Injection ❌

```

class Car {
    Engine engine = new Engine();
}

```

With Dependency Injection ✅

```

class Car {
    Engine engine;
    Car(Engine engine) {
        this.engine = engine;
    }
}

```

Types of Dependency Injection

Type	Description
Constructor Injection	Dependency passed through constructor

Setter Injection	Dependency passed using setter
Field Injection	Dependency injected directly into field

Spring Boot Example

```
@Service
class UserService {
    @Autowired
    UserRepository repository;
}
```

Spring automatically injects dependency. **Benefits:** Loose coupling, Easier testing, Better maintainability, Flexible architecture.

7. SOLID Principles

SOLID is a set of 5 principles for writing clean and maintainable code.

S → Single Responsibility Principle (SRP)

Definition: A class should have only ONE reason to change.

Bad Example ❌ (One class doing multiple tasks: Salary + DB)

Good Example ✅ (SalaryService + EmployeeRepository)

O → Open/Closed Principle (OCP)

Definition: Software should be Open for extension, Closed for modification.

L → Liskov Substitution Principle (LSP)

Definition: Child class should properly replace parent class. (Example: Ostrich inherits Bird but cannot fly).

I → Interface Segregation Principle (ISP)

Definition: Clients should not be forced to implement methods they do not use. (Example: Robot shouldn't implement eat() from Worker interface).

D → Dependency Inversion Principle (DIP)

Definition: Depend on abstraction, not concrete classes.

8. Explain MVC Architecture

Definition

MVC stands for: Model, View, Controller. It separates application logic.

Components

Model: Handles data/business logic.

View: UI/Frontend.

Controller: Handles requests.

Flow: User → Controller → Model → Database | View → User

Example in Spring Boot

Model: `class User { ... }`

Controller: `@RestController class UserController { ... }`

View: HTML, JSP, Thymeleaf.